

UNIVERSIDAD PRIVADA ANTENOR ORREGO
FACULTAD DE INGENIERÍA
PROGRAMA DE ESTUDIO DE INGENIERÍA DE
SISTEMAS E INTELIGENCIA ARTIFICIAL



"PLATAFORMAS MIDDLEWARE – SII_S04"

Curso: Sistemas de Información Integrados

Proyecto: Sistema de Gestión Hotelera con IA – Grupo 5 (módulo CRM)

Integrantes:

Aguilar Idrogo, Clever Josué

Carranza Jacinto, Juan Diego

Castillo Pezo, Mateo Salvador Vincenzo

Romero Uriol, Jeffry Anderson

Vidal Rodríguez, Fabrizio

Profesor: Ing. Carlos Alfredo Mendoza Corpus

Repositorio de la solución: <https://github.com/JeffryRU/SII-S04-Middleware>

Trujillo - PERÚ

2026

Índice

1. Introducción.....	3
2. Caso 1: programa de mensajería con middleware.....	4
2.1. Objetivo y tipo de middleware.....	4
2.2. Arquitectura de la solución.....	4
2.3. Protocolo de mensajería.....	5
2.4. Componentes del middleware implementados.....	6
2.5. Transferencia de archivos.....	6
2.6. Ejecución y pruebas.....	7
2.7. Limitaciones y mejoras posibles.....	8
3. Caso 2: conexión entre Microsoft Excel y la base de datos.....	9
3.1. Base de datos utilizada.....	9
3.2. Tipo de middleware: middleware de acceso a datos.....	9
3.3. Procedimiento de conexión.....	11
3.4. Resultado.....	17
3.5. Alternativa mediante ODBC.....	17
4. Caso 3: formulario de consulta de palabras en Python.....	18
4.1. Descripción.....	18
4.2. Palabras incluidas.....	18
4.3. Funcionamiento.....	19
5. Caso 4: plataformas de pago en el Perú – Yape.....	21
5.1. ¿Qué es Yape?.....	21
5.2. Infraestructura.....	21
5.3. Middleware.....	22
5.4. Patrón arquitectónico.....	23
5.5. Otras características.....	24
5.6. Relación con el curso.....	25
6. Caso 5: ensayo.....	26
6.1. Introducción.....	26
6.2. Síntesis del capítulo 1: el poder de las APP en la infraestructura social.....	26
6.3. Síntesis del capítulo 2: proyectos resilientes y centrados en las personas.....	27
6.4. Opinión.....	28
6.5. Relación con los sistemas de información integrados y con la carrera.....	28
6.6. Conclusiones.....	29
7. Enlace y contenido de la solución.....	30
Referencias.....	31

1. Introducción

La sesión 04 del curso presentó el middleware como la capa de software que permite que aplicaciones, bases de datos y servicios distintos se comuniquen como si formaran un solo sistema, además del bus de servicios empresarial (ESB) y las plataformas de integración (Mendoza Corpus, 2026). Este informe documenta los cinco casos solicitados en la actividad y los relaciona con esos conceptos.

El **caso 1** implementa en Python un sistema de mensajería con transferencia de archivos basado en un servidor intermediario (middleware orientado a mensajes). El **caso 2** conecta Microsoft Excel con la base de datos de Access *VERDULEROS.mdb* y explica el middleware de acceso a datos que lo hace posible (ODBC, OLE DB y Power Query). El **caso 3** es un formulario en Python que busca el significado de las palabras de la sopa de letras del Laboratorio 03 y de los términos nuevos de la sesión S04. El **caso 4** analiza la plataforma de pagos Yape: su infraestructura, su middleware y su patrón arquitectónico. El **caso 5** es el ensayo sobre los capítulos 1 y 2 del documento *PPP Américas 2025. Perú: Alianzas que mejoran vidas* (Lembo et al., 2025).

Los programas, el libro de Excel y las capturas se encuentran en el repositorio del grupo: <https://github.com/JeffryRU/SII-S04-Middleware>. En la sección 7 se describe su contenido y la forma de ejecutarlos.

2. Caso 1: programa de mensajería con middleware

2.1. Objetivo y tipo de middleware

El objetivo fue crear un programa que permita la comunicación entre varias personas mediante mensajes y transferencia de archivos. Para que sea un middleware y no una simple conexión punto a punto, los clientes nunca se comunican directamente entre sí: todos se conectan a un **servidor intermediario** (*broker*) que recibe cada mensaje, lo valida, decide a quién entregarlo y lo reenvía.

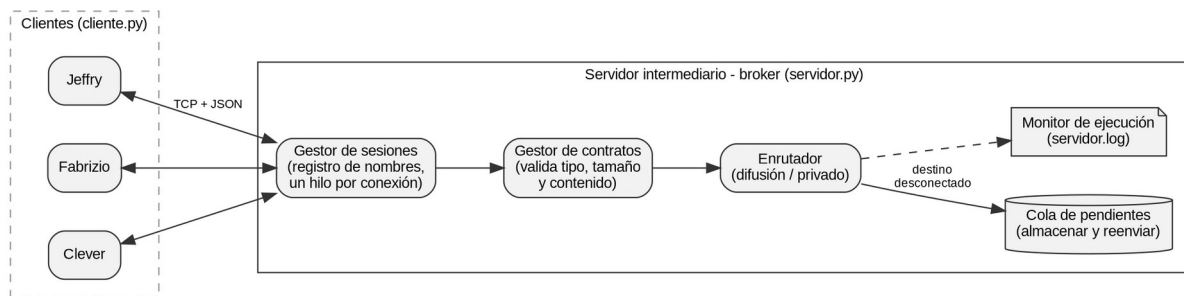
Por eso la solución corresponde a un **middleware orientado a mensajes (MOM)**. En la sesión se le define como el middleware que permite que los componentes de una aplicación intercambien mensajes y que, además de transformarlos, gestiona su enrutamiento para que lleguen a los componentes adecuados y en el orden apropiado; sus ejemplos típicos son las colas y los agentes de mensajes (Mendoza Corpus, 2026). En un MOM el emisor no necesita conocer la dirección de red ni el estado del receptor, porque el intermediario se encarga de la entrega; a esto se le llama desacoplamiento (Curry, 2004). En el programa, cada persona solo conoce la dirección del servidor y el nombre de los demás usuarios.

2.2. Arquitectura de la solución

La Figura 1 muestra la arquitectura. Cada cliente abre una conexión TCP con el servidor. Dentro del servidor, cada mensaje pasa por el gestor de sesiones, el gestor de contratos y el enrutador; si el destinatario de un mensaje privado no está conectado, el mensaje se guarda en una cola y se entrega cuando esa persona vuelve a conectarse (almacenar y reenviar). Toda la actividad queda registrada en un archivo de bitácora.

Figura 1

Arquitectura del sistema de mensajería



Nota. Elaboración propia. Los nombres de los componentes siguen la clasificación de la sesión S04 (Mendoza Corpus, 2026).

La solución tiene tres archivos: *protocolo.py* define el formato de los mensajes, *servidor.py* implementa el intermediario y *cliente.py* es la interfaz gráfica que usa cada persona. Solo se utilizó la biblioteca estándar de Python (*socket*, *threading*, *json*, *base64* y *tkinter*), así que no hay que instalar paquetes adicionales.

2.3. Protocolo de mensajería

TCP entrega los datos como un flujo continuo de bytes, sin separar un mensaje del siguiente. Por eso cada mensaje se envía como una **trama**: un encabezado de 4 bytes con la longitud del contenido, seguido del mensaje en formato JSON codificado en UTF-8. El receptor lee primero los 4 bytes, sabe cuántos bytes más debe leer y así reconstruye cada mensaje completo aunque la red lo entregue en partes. JSON cumple el papel del marco de mensajería común descrito en la sesión. La Tabla 1 resume los tipos de mensaje.

Tabla 1

Tipos de mensaje del protocolo

Tipo	Sentido	Campos principales	Uso
registro	Cliente → servidor	nombre	Identificarse al conectarse.
bienvenida	Servidor → cliente	nombre, usuarios	Confirmar el registro y enviar la lista de conectados.
texto	Ambos	de, para, texto, hora	Mensaje a todos (para = "*") o privado (para = nombre).
archivo	Ambos	de, para, nombre_archivo, tamaño, datos	Transferencia de un archivo codificado en Base64.
usuarios	Servidor → cliente	usuarios	Actualizar la lista de conectados.

Tipo	Sentido	Campos principales	Uso
sistema	Servidor → cliente	texto, hora	Avisos: ingresos, salidas y mensajes en cola.
error	Servidor → cliente	texto	Rechazo de un mensaje que incumple las reglas.
salir	Cliente → servidor	—	Cerrar la sesión de forma ordenada.

Nota. Elaboración propia.

2.4. Componentes del middleware implementados

La sesión S04 enumera los componentes estándar de un middleware. La Tabla 2 muestra dónde aparece cada uno en el código.

Tabla 2

Correspondencia entre los componentes del middleware y el código

Componente (S04)	Implementación	Detalle
Interfaz del cliente	cliente.py y protocolo.py	Ventana Tkinter con destino, mensaje, envío de archivos y lista de conectados. Un hilo receptor coloca los mensajes en una cola y la interfaz los muestra cada 100 ms, para que la ventana no se congele.
Gestor de sesiones	Servidor.atender()	Registra un nombre único por conexión, crea un hilo por cliente y usa un candado de envío por cliente para que dos mensajes no se mezclen en el mismo socket.
Gestor de contratos	Servidor.validar()	Acepta solo los tipos permitidos, rechaza textos vacíos o de más de 4000 caracteres y archivos de más de 10 MB. Si una regla se incumple, devuelve un mensaje de error, como la excepción descrita en la sesión.
Enrutamiento	Servidor.enrutar()	Destino "*": difusión a todos menos al emisor. Nombre conectado: entrega privada. Nombre desconectado: cola. Nombre inexistente: error.
Cola de mensajes	Servidor.pendientes	Una cola por usuario para entregar mensajes privados cuando vuelve a conectarse.
Monitor de tiempo de ejecución	Servidor.registrar()	Escribe conexiones, desconexiones, envíos y errores con fecha y hora en servidor.log.
Marco de mensajería	protocolo.py	Tramas con longitud + JSON en UTF-8.

Nota. Elaboración propia.

2.5. Transferencia de archivos

Para enviar un archivo, el cliente lo lee en modo binario, lo codifica en Base64 y lo envía en un mensaje de tipo *archivo* junto con su nombre y tamaño; el envío se hace en un hilo aparte para no bloquear la ventana. El servidor lo valida y lo enruta igual que un texto, a todos o a un usuario. El receptor decodifica el contenido y lo guarda en la carpeta *recibidos/*<su

nombre>/. Para evitar que un nombre malicioso escriba fuera de esa carpeta, solo se usa el nombre base del archivo, y si ya existe un archivo con el mismo nombre se agrega un número al final. Base64 aumenta el tamaño en aproximadamente un tercio, por eso se fijó un límite de 10 MB por archivo.

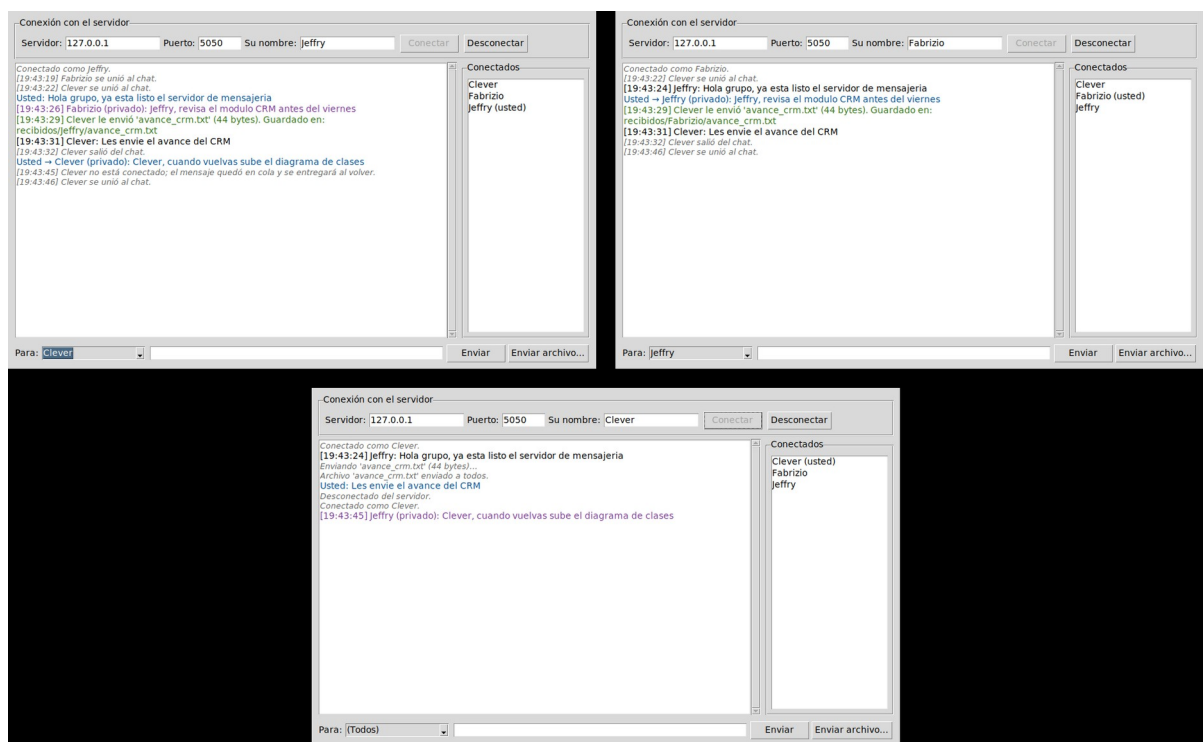
2.6. Ejecución y pruebas

Para ejecutar el sistema se inicia primero el servidor con *python servidor.py* (escucha en el puerto 5050) y luego cada persona abre *python cliente.py*, escribe la dirección IP del servidor, el puerto y su nombre, y presiona **Conectar**. Si todos están en la misma computadora se usa la dirección 127.0.0.1; en una red local se usa la IP de la computadora que ejecuta el servidor.

La Figura 2 muestra una prueba con tres usuarios. Jeffry envía un mensaje a todos; Fabrizio le responde en privado; Clever envía el archivo *avance_crm.txt* a todos, que queda guardado en la carpeta de cada receptor. Después Clever se desconecta, Jeffry le deja un mensaje privado que el servidor pone en cola, y cuando Clever vuelve a conectarse lo recibe.

Figura 2

Prueba del sistema de mensajería con tres usuarios



Nota. Captura de la ejecución de prueba con el servidor y tres clientes en un mismo equipo. Los mensajes en morado son privados y los verdes, archivos recibidos.

La bitácora del servidor registró la misma secuencia:

```
[2026-09-22 19:43:11] Servidor de mensajería escuchando en 0.0.0.0:5050
[2026-09-22 19:43:15] Conexión de Jeffry desde 127.0.0.1:48516
[2026-09-22 19:43:19] Conexión de Fabrizio desde 127.0.0.1:48528
[2026-09-22 19:43:22] Conexión de Clever desde 127.0.0.1:48530
[2026-09-22 19:43:24] Jeffry -> todos: texto
[2026-09-22 19:43:26] Fabrizio -> Jeffry: texto
[2026-09-22 19:43:29] Clever -> todos: archivo 'avance_crm.txt' (44 B)
[2026-09-22 19:43:31] Clever -> todos: texto
[2026-09-22 19:43:32] Desconexión de Clever
[2026-09-22 19:43:45] Jeffry -> Clever: texto (en cola)
[2026-09-22 19:43:46] Conexión de Clever desde 127.0.0.1:54892
```

Además de la prueba con la interfaz, se verificaron con un script los casos de error. La Tabla 3 resume los resultados.

Tabla 3

Pruebas del sistema de mensajería

Caso de prueba	Resultado esperado	Resultado
Mensaje a todos	Lo reciben todos menos el emisor	Correcto
Mensaje privado	Lo recibe solo el destinatario	Correcto
Envío de archivo	El receptor obtiene el archivo con el mismo contenido	Correcto
Mensaje vacío	Error: "El texto no puede estar vacío"	Correcto
Destinatario inexistente	Error: "El usuario no existe"	Correcto
Nombre ya en uso	Se rechaza la conexión	Correcto
Privado a un usuario desconectado	Se guarda en cola y se entrega al reconectarse	Correcto

Nota. Elaboración propia.

2.7. Limitaciones y mejoras posibles

- La comunicación no está cifrada. Se podría agregar TLS con el módulo *ssl* de Python, que la sesión menciona como mecanismo para proteger la transferencia de datos.
- Los usuarios se identifican solo con un nombre, sin contraseña.
- La cola de pendientes vive en memoria: si el servidor se reinicia, esos mensajes se pierden. Un MOM comercial, como RabbitMQ, guarda las colas en disco.
- Hay un único servidor. Para muchos usuarios se necesitarían varios intermediarios coordinados, como ocurre con Apache Kafka.

3. Caso 2: conexión entre Microsoft Excel y la base de datos

3.1. Base de datos utilizada

La base de datos *VERDULEROS.mdb* es un archivo de Microsoft Access en formato Jet 4.0 (Access 2000-2003) con el registro de ventas de una distribuidora de frutas y verduras. Contiene cuatro tablas y una consulta guardada que une las ventas con los vendedores, los productos y los grupos (Tabla 4).

Tabla 4

Objetos de la base de datos VERDULEROS.mdb

Objeto	Campos	Registros
Grupos	IdGrupo, NombreGrupo	3
Productos	IdProducto, NomProducto, IdGrupo, Precio	15
Vendedores	IdVendedor, NombreVendedor, FechaAlta, NIF, FechaNac, Direccion, Poblacion, CodPostal, Telefon, EstalCivil, Guap@	14
Ventas	Cod Vendedor, Cod Producto, Fecha, Kilos	5001
Consulta de ventas (consulta)	NombreVendedor, Fecha, Kilos, Precio, NombreGrupo	5001

Nota. Elaboración propia a partir del esquema de la base de datos.

3.2. Tipo de middleware: middleware de acceso a datos

Excel no sabe leer por sí mismo el formato interno de un archivo *.mdb*. Entre la hoja de cálculo y la base de datos se ubica un **middleware de datos o de bases de datos**, que según la sesión es el tipo de middleware que simplifica el acceso y la interacción con las bases de datos de *back-end* (Mendoza Corpus, 2026). Excel solo pide tablas o resultados de consultas y el middleware se encarga de abrir el archivo, interpretar su estructura y devolver filas y columnas. En Windows este middleware tiene tres piezas principales.

ODBC (Open Database Connectivity).

ODBC es una interfaz de programación estándar para acceder a sistemas gestores de bases de datos. Su objetivo es que una aplicación pueda consultar distintas bases de datos con el mismo código, sin depender del producto. Su arquitectura tiene cuatro capas: la aplicación, el **administrador de controladores (Driver Manager)**, que carga el controlador adecuado; el **controlador (driver)**, que traduce las llamadas ODBC al lenguaje de cada base de datos; y el

origen de datos (Microsoft, s. f.-b). Para Access existe el controlador *Microsoft Access Driver* (*.mdb, *.accdb).

OLE DB y el motor ACE.

OLE DB es la tecnología de acceso a datos de Microsoft basada en componentes COM. Funciona con **proveedores**: cada proveedor sabe leer un tipo de fuente de datos y la expone con una interfaz común. Para Access, el proveedor es *Microsoft.ACE.OLEDB.12.0*, que pertenece al Access Database Engine (ACE), sucesor del antiguo motor Jet. Este proveedor viene con Microsoft 365 y también se puede instalar por separado; su arquitectura (32 o 64 bits) debe coincidir con la de la aplicación que lo usa, porque de lo contrario aparece el error "*The 'Microsoft.ACE.OLEDB.12.0' provider is not registered*" (Microsoft, s. f.-a).

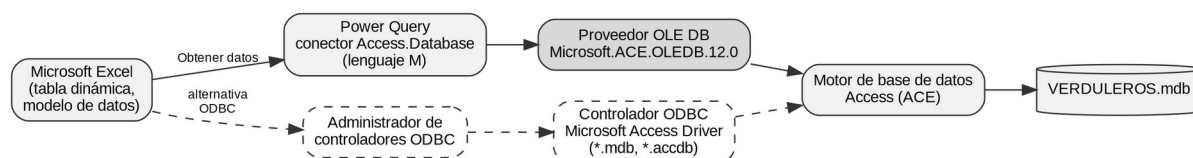
Power Query.

Power Query es el motor de extracción, transformación y carga de datos integrado en Excel (**Datos → Obtener y transformar datos**). Cada conexión se guarda como una consulta escrita en lenguaje M. Su conector para Access usa la función *Access.Database*, que a su vez se apoya en el proveedor OLE DB de ACE (Microsoft, s. f.-a). Power Query también actúa como middleware porque permite filtrar, combinar o transformar los datos antes de cargarlos, igual que la transformación de datos que la sesión atribuye a las plataformas de integración.

La Figura 3 muestra cómo se relacionan estas piezas. En este trabajo se usó la ruta de Power Query y OLE DB (línea continua); la ruta ODBC (línea discontinua) se describe en la sección 3.5 como alternativa.

Figura 3

Capas de middleware entre Excel y la base de datos de Access



Nota. Elaboración propia a partir de Microsoft (s. f.-a, s. f.-b).

Tabla 5

Comparación de las tecnologías de acceso a datos

Aspecto	ODBC	OLE DB (ACE)	Power Query
Qué es	API estándar de acceso a bases de datos	Tecnología COM de Microsoft basada en proveedores	Motor ETL de Excel y Power BI
Pieza clave para Access	Microsoft Access Driver (*.mdb, *.accdb)	Microsoft.ACE.OLEDB.12.0	Función Access.Database (lenguaje M)
Uso en Excel	Desde ODBC o Microsoft Query, con un DSN	Conexiones heredadas y uso interno de Power Query	Datos → Obtener datos
Ventaja	Multiplataforma y sirve para muchos motores	Acceso nativo y rápido a Access	Transforma datos y se actualiza con un clic

Nota. Elaboración propia a partir de Microsoft (s. f.-a, s. f.-b).

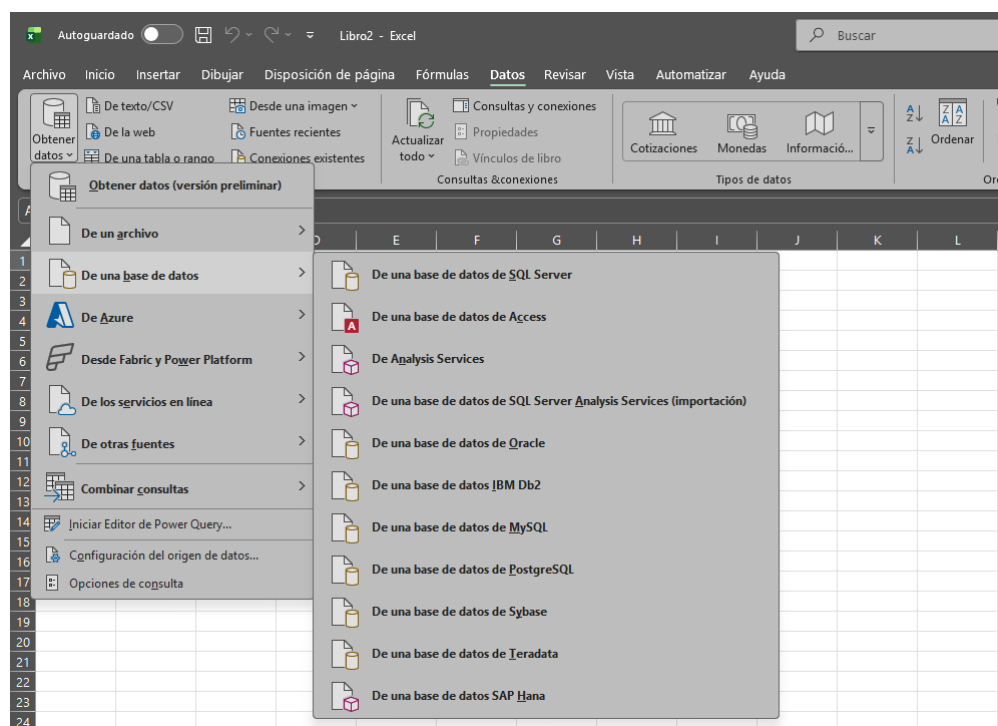
3.3. Procedimiento de conexión

La conexión se realizó en Microsoft Excel de Microsoft 365 para Windows, con los siguientes pasos.

1. En una hoja en blanco, abrir la pestaña **Datos** y elegir **Obtener datos** → **De una base de datos** → **De una base de datos de Microsoft Access** (Figura 4).

Figura 4

Menú Obtener datos de Excel

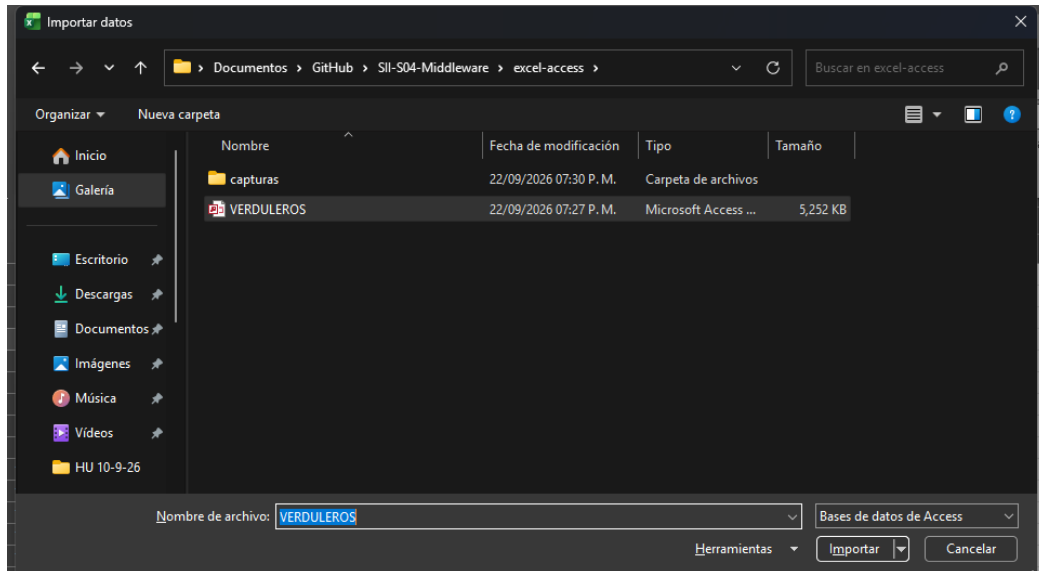


Nota. Captura propia en Microsoft Excel.

2. En la ventana **Importar datos**, ubicar el archivo *VERDULEROS.mdb*, seleccionarlo y presionar **Importar** (Figura 5).

Figura 5

Selección del archivo de la base de datos

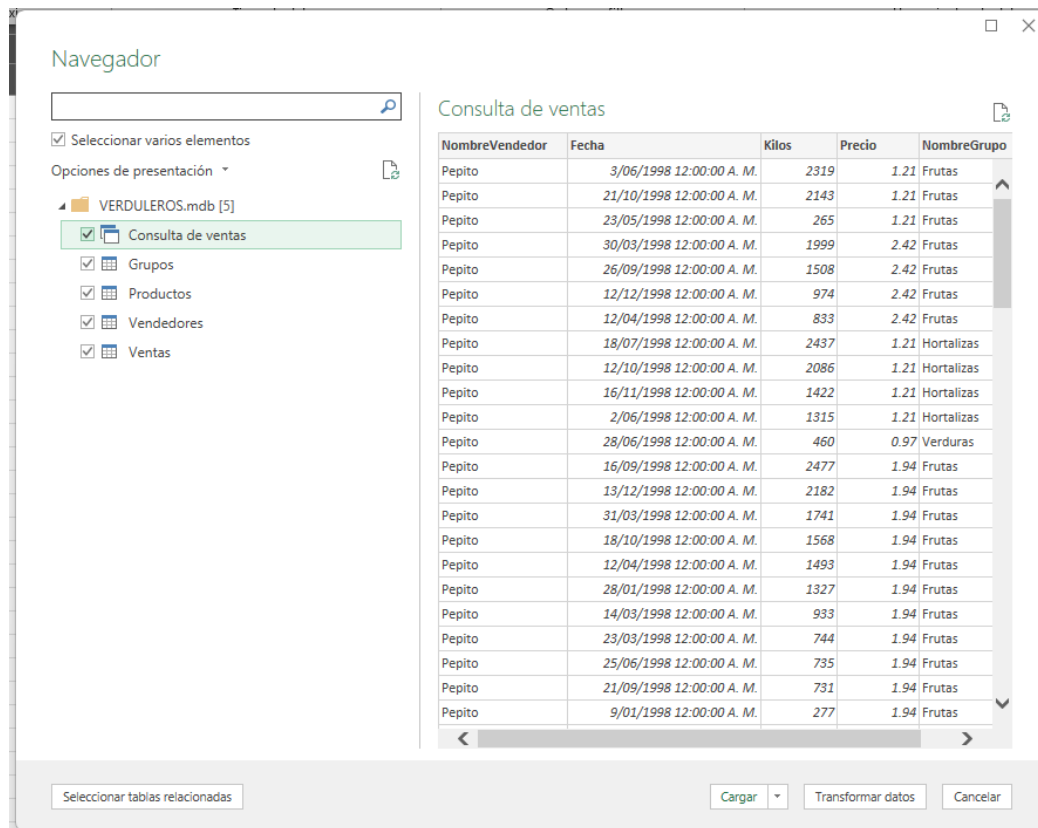


Nota. Captura propia en Microsoft Excel.

3. En el **Navegador**, activar **Seleccionar varios elementos** y marcar las cuatro tablas y la *Consulta de ventas*. Al seleccionar un objeto, la parte derecha muestra una vista previa de sus datos (Figura 6).

Figura 6

Navegador de Power Query con las tablas de VERDULEROS.mdb

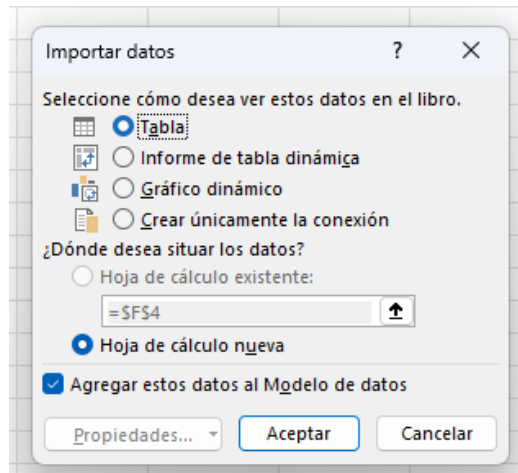


Nota. Captura propia en Microsoft Excel.

4. Presionar **Cargar**. Con la opción **Cargar en...** se puede elegir el destino: tabla, informe de tabla dinámica, gráfico dinámico o solo conexión, y si los datos se agregan al **modelo de datos** (Figura 7). En este caso los cinco objetos se cargaron al modelo de datos; el panel **Consultas y conexiones** confirmó 3, 15, 14, 5001 y 5001 filas (Figura 8, derecha).

Figura 7

Opciones de importación de datos



Nota. Captura propia en Microsoft Excel.

5. Para analizar los datos, elegir **Insertar** → **Tabla dinámica** → **Desde el modelo de datos**. Se colocó *NombreVendedor* en filas, *NombreGrupo* en columnas y la suma de *Kilos* en valores (Figura 8). El total general, 6 865 490 kilos, coincide con la suma de la tabla *Ventas* calculada directamente sobre la base de datos, lo que confirma que la importación fue completa.

Figura 8

Tabla dinámica de kilos vendidos por vendedor y grupo

The screenshot shows the Microsoft Excel interface. The main window displays a PivotTable with the following data:

	Frutas	Hortalizas	Verduras	Total general
Amadeu	1701			1701
Antonio	253019	144245	86571	483835
Carla	275364	154873	127162	557399
Carlos	247870	151558	105312	504740
Carmen	285296	133291	128187	546774
Enrique	277809	148564	98229	524602
Federico	240583	142237	113822	496642
Fran	296256	129963	117464	543683
Gloria	312739	163779	108556	585074
Monica	269693	145743	126690	542126
Pepito	273200	146659	111943	531802
Quima	245115	143153	134812	523080
Ramon	233380	167876	91716	492972
Rosa	279338	137449	114273	531060
Total general	3491363	1909390	1464737	6865490

On the right side, the 'Consultas y conexiones' task pane is open, showing a list of queries and their row counts:

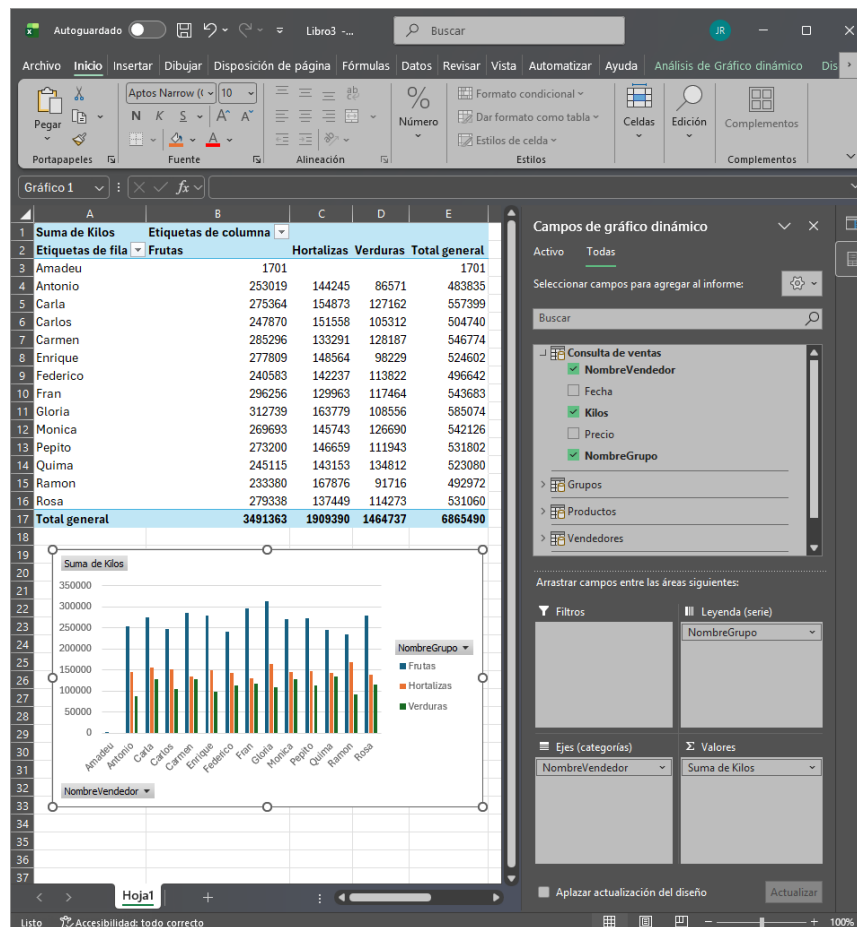
- Grupos: Se cargaron 3 filas.
- Productos: Se cargaron 15 filas.
- Vendedores: Se cargaron 14 filas.
- Ventas: Se cargaron 5,001 filas.
- Consulta de ventas: Se cargaron 5,001 filas.

Nota. Captura propia en Microsoft Excel. A la derecha, el panel Consultas y conexiones con las filas cargadas.

- Con una celda de la tabla dinámica seleccionada, presionar **Alt + F1** para crear un gráfico dinámico (Figura 9).

Figura 9

Gráfico dinámico conectado a la base de datos



Nota. Captura propia en Microsoft Excel.

7. Guardar el libro como *Conexion_Excel_VERDULEROS.xlsx*. Si la base de datos cambia, basta con **Datos** → **Actualizar todo** para volver a leerla.

Power Query genera automáticamente una consulta en lenguaje M por cada objeto importado. Por ejemplo, la consulta de la tabla *Ventas* es:

```
let
    Origen = Access.Database(File.Contents("C:\Users\JeffryRU\Documents\GitHub\
        SII-S04-Middleware\excel-access\VERDULEROS.mdb"),
        [CreateNavigationProperties=true]),
    _Ventas = Origen{[Schema="",Item="Ventas"]}[Data]
in
    _Ventas
```

La ruta del archivo queda guardada en la consulta. Si el libro se abre en otra computadora, se debe actualizar en **Datos** → **Obtener datos** → **Configuración de origen de datos** → **Cambiar origen** antes de actualizar.

3.4. Resultado

La tabla dinámica muestra que el grupo *Frutas* concentra 3 491 363 kilos, seguido de *Hortalizas* (1 909 390) y *Verduras* (1 464 737). Los vendedores con más kilos son Gloria (585 074), Carla (557 399) y Carmen (546 774). Lo importante para el curso es que Excel trabaja con datos que siguen viviendo en Access: el libro no copia la base de datos a mano, sino que mantiene una conexión que se puede actualizar.

3.5. Alternativa mediante ODBC

La misma conexión se puede hacer con ODBC. No fue necesaria en este trabajo, pero el procedimiento es el siguiente:

1. Abrir el **Administrador de orígenes de datos ODBC** de Windows (odbcad32) en la versión de 64 bits, que coincide con la de Excel.
2. En **DSN de usuario**, elegir **Agregar**, seleccionar *Microsoft Access Driver (*.mdb, *.accdb)*, dar un nombre al origen (por ejemplo, VERDULEROS) y seleccionar el archivo *.mdb*.
3. En Excel, elegir **Datos** → **Obtener datos** → **De otras fuentes** → **Desde ODBC**, escoger el DSN creado y continuar en el Navegador igual que en la sección 3.3.

4. Caso 3: formulario de consulta de palabras en Python

4.1. Descripción

El programa *diccionario.py* es un formulario de escritorio hecho con Tkinter. El usuario escribe una palabra y el programa muestra su significado y de qué fuente proviene. Funciona sin conexión a Internet porque el listado de palabras y sus significados está dentro del mismo archivo, en un diccionario de Python. Sus funciones son:

- **Búsqueda exacta sin distinguir mayúsculas ni tildes:** antes de comparar, el texto se normaliza con el módulo *unicodedata* (por ejemplo, *analisis* encuentra ANÁLISIS).
- **Listado filtrable:** la lista de la izquierda se reduce mientras se escribe y se puede filtrar por fuente (todas, Laboratorio 03 o S04). Al hacer clic en una palabra se muestra su significado.
- **Sugerencias:** si la palabra no existe, el programa propone las más parecidas con el módulo *difflib*.
- **Consulta en línea opcional:** para palabras fuera del listado, el botón **Buscar en Wikipedia** consulta el resumen del artículo en Wikipedia en español mediante su API REST. Es la única función que necesita Internet.

4.2. Palabras incluidas

El listado tiene 101 palabras. Las primeras 49 son las de la sopa de letras *Sistemas verticales y horizontales* del Laboratorio 03, con los significados que el grupo elaboró en esa actividad (Tabla 6).

Tabla 6

Palabras del Laboratorio 03

ACCESO	ACTUALIZACIÓN	ADAPTACIÓN	AGILIDAD
ALMACENAMIENTO	ANÁLISIS	AUTOMATIZACIÓN	BASE DE DATOS
BENEFICIOS	CLÍNICAS	COLABORACIÓN	COMPATIBILIDAD
COMPROMISO	COMUNICACIÓN	CONSOLIDACIÓN	CONSULTORÍA
COSTE	CRECIMIENTO	DECISIONES	DESARROLLO
DIGITAL	DOCUMENTACIÓN	EFICACIA	ESCALABILIDAD
ESTRATEGIA	HISTORIAS	HOJA DE CÁLCULO	IMPLEMENTACIÓN
INFORMES	INNOVACIÓN	INTEGRADOS	INTERFAZ
MÓDULOS	OBJETIVOS	OPTIMIZACIÓN	PARÁMETROS
PARTNER	PERSONALIZACIÓN	PLANES	PREDICCIÓN
PROYECTO	REDUCCIÓN	RENTABILIDAD	REPORTES

RESULTADOS	SEGUIMIENTO	SEGURIDAD	SOCIOS
TRANSFORMACIÓN			

Nota. Palabras de la sopa de letras del Laboratorio 03.

Del documento S04_SII se seleccionaron 52 términos que el grupo no conocía o no dominaba: principalmente siglas (ORB, CORBA, TPM, iPaaS, MWaaS, WSDL, CAGR), tipos de middleware y conceptos de nube y despliegue que la presentación menciona sin definir (Kubernetes, contenedor, *serverless*, DevSecOps). Sus significados se redactaron a partir de la presentación y se completaron con las definiciones usuales de cada término (Tabla 7).

Tabla 7

Términos seleccionados de la sesión S04

MIDDLEWARE	MOM	RPC	ORB
CORBA	TPM	MIDDLEWARE TRANSACCIONAL	MIDDLEWARE DE PORTAL
ESB	SOA	iPaaS	PaaS
SaaS	MWaaS	API	API GATEWAY
REST	SOAP	WSDL	JSON
XML	ENDPOINT	APACHE KAFKA	ASÍNCRONO
ENRUTAMIENTO	ACOPLAMIENTO	ORQUESTACIÓN	CONECTOR
GESTOR DE CONTRATOS	GESTOR DE SESIONES	DATA PIPELINE	SISTEMA HEREDADO
MAINFRAME	MICROSERVICIOS	CONTENEDOR	KUBERNETES
NATIVO DE LA NUBE	NUBE HÍBRIDA	MULTINUBE	ON-PREMISES
CSP	DEVOPS	DEVSECOPS	CI/CD
TLS	SERVERLESS	LOW-CODE	CONFIANZA CERO
SILOS DE DATOS	GOBERNANZA	CAGR	INTEROPERABILIDAD

Nota. Elaboración propia a partir de Mendoza Corpus (2026).

4.3. Funcionamiento

Para ejecutar el programa se usa *python diccionario.py*. La Figura 10 muestra tres situaciones: una palabra del Laboratorio 03 encontrada, el filtro por fuente con la lista reducida a los términos que contienen "mid", y una búsqueda sin resultado que sugiere la palabra más parecida y habilita la consulta en Wikipedia.

Figura 10

Formulario de consulta de palabras

The figure displays three sequential screenshots of a web application titled "Diccionario de Sistemas de Información Integrados". Each screenshot shows a search interface with a search bar, buttons for "Buscar", "Limpiar", and "Buscar en Wikipedia", and two main panels: "Listado" and "Significado".

Screenshot 1: The search bar contains the word "escalabilidad". The "Listado" panel shows a single result, "ESCALABILIDAD", under the "Todas" category. The "Significado" panel displays the definition: "Capacidad del sistema de soportar más usuarios, datos o sedes sin perder rendimiento." and attributes the source to "Laboratorio 03 - Sopa de letras".

Screenshot 2: The search bar contains the word "mid". The "Listado" panel shows three results: "S04 - Plataformas middleware", "MIDDLEWARE", "MIDDLEWARE DE PORTAL", and "MIDDLEWARE TRANSACCIONAL". The "Significado" panel displays the definition for "MIDDLEWARE DE PORTAL": "Middleware que integra contenido y funciones de varias aplicaciones en una sola pantalla o aplicación compuesta." and attributes the source to "S04 - Plataformas middleware".

Screenshot 3: The search bar contains the word "orquestracion de contenedores". The "Listado" panel shows no results. The "Significado" panel displays a message: "No encontrada. La palabra no está en el listado. ¿Quiso decir?: ORQUESTACIÓN. Puede consultarla en Wikipedia con el botón «Buscar en Wikipedia»."

Nota. Captura de la ejecución del programa diccionario.py.

5. Caso 4: plataformas de pago en el Perú – Yape

5.1. ¿Qué es Yape?

Yape es una billetera digital creada en 2016 por el Banco de Crédito del Perú (BCP) que permite enviar y recibir dinero usando solo el número de celular o un código QR. Al inicio solo servía para transferencias entre clientes del BCP; luego se abrió a usuarios de otros bancos y a personas sin cuenta bancaria, lo que la convirtió en una herramienta de inclusión financiera (Universidad Tecnológica del Perú [UTP], 2024). Hoy pertenece al ecosistema de Credicorp, junto con iO y Tenpo.

Su escala explica muchas de sus decisiones técnicas. En el primer trimestre de 2026, Yape alcanzó 16,4 millones de usuarios activos mensuales, alrededor del 82 % de la población económicamente activa del Perú, con 67 transacciones al mes por usuario activo (Credicorp, 2026). A fines de 2024 el BCP reportaba cerca de 27 millones de transacciones diarias en promedio (Microsoft News Center Latinoamérica, 2024). Desde agosto de 2023 opera también en Bolivia, donde supera los 3 millones de usuarios (Infobae, 2025). Además de transferencias, ofrece pagos con QR a negocios, pago de servicios, recargas, préstamos y compras por Internet.

Nota sobre las fuentes. Yape y el BCP no publican documentación técnica detallada de su arquitectura interna. Por eso, en las secciones siguientes se distingue entre lo que está **documentado** en fuentes públicas (datos de la empresa, del BCRP, de Microsoft y de las pasarelas de pago) y lo que es una **reconstrucción razonada** a partir de esos datos y de los patrones que usan las plataformas de pago de esta escala.

5.2. Infraestructura

- **Nube híbrida (documentado).** En noviembre de 2024 el BCP anunció una inversión de más de S/ 2 500 millones en cinco años para modernizar su infraestructura tecnológica con Microsoft y Kyndryl, adoptando Microsoft Azure en un entorno de nube híbrida que combina la nube pública con sus centros de datos (Microsoft News Center Latinoamérica, 2024). Yape, como producto del BCP, se apoya en esa infraestructura.
- **Aplicaciones móviles (documentado).** El canal principal es la app para Android e iOS, que concentra el registro, los *yapeos*, el escaneo de QR y la generación de códigos de aprobación.

- **Núcleo bancario (documentado/inferido).** Las cuentas y el dinero de los usuarios se registran en los sistemas del banco. Un banco de la antigüedad del BCP mantiene sistemas centrales heredados, que la sesión describe como el caso típico que exige middleware para integrarse con aplicaciones móviles (Mendoza Corpus, 2026).
- **Cámara de Compensación Electrónica – CCE (documentado).** Desde la interoperabilidad, los pagos con otras billeteras y bancos se liquidan en tiempo real a través de la CCE, que actúa como punto central de conexión entre participantes (Gestión, s. f.).
- **Pasarelas de pago (documentado).** Para compras por Internet, Yape se integra con Culqi, Izipay y Niubiz, que exponen APIs a los comercios (Culqi, s. f.; Yape, s. f.).

5.3. Middleware

Yape usa varios tipos de middleware a la vez, cada uno en una frontera distinta del sistema. La Tabla 8 los resume con el tipo de evidencia disponible.

Tabla 8

Tipos de middleware presentes en el funcionamiento de Yape

Tipo de middleware (S04)	Uso en Yape	Evidencia
Middleware de API	Las pasarelas exponen APIs REST con JSON. El comercio pide al cliente su celular y el código de aprobación que genera la app (válido 2 minutos); con ellos crea un token y luego el cargo, hasta S/ 2 000 por operación.	Documentado (Culqi, s. f.)
Plataforma de integración / conmutador de pagos	La CCE conecta a los participantes directos (bancos, emisores de dinero electrónico, cooperativas); fintech, bigtech y operadoras se conectan como participantes indirectos a través de un participante directo.	Documentado (Gestión, s. f.)
Middleware de integración con sistemas heredados (ESB o iPaaS)	Traduce las operaciones de la app a los formatos del núcleo bancario, como en el ejemplo tienda → middleware → ERP de la sesión.	Inferido
Middleware orientado a mensajes y eventos	Notificaciones al receptor, actualización de saldos, alertas y análisis de fraude se procesan como eventos en colas, sin bloquear el pago.	Inferido
Middleware transaccional	Garantiza que el débito al emisor y el crédito al receptor se completen juntos o se reviertan.	Inferido (requisito de todo sistema de pagos)

Nota. Elaboración propia a partir de Culqi (s. f.), Gestión (s. f.) y Mendoza Corpus (2026).

El caso de las compras en línea muestra bien el papel del middleware: el comercio nunca accede a la cuenta del cliente ni al sistema del banco. Solo conversa con la API de la pasarela, que valida el código de aprobación con Yape y devuelve un resultado. Es la misma

idea del caso 1: las partes no se conectan entre sí, sino a través de un intermediario que valida y enruta.

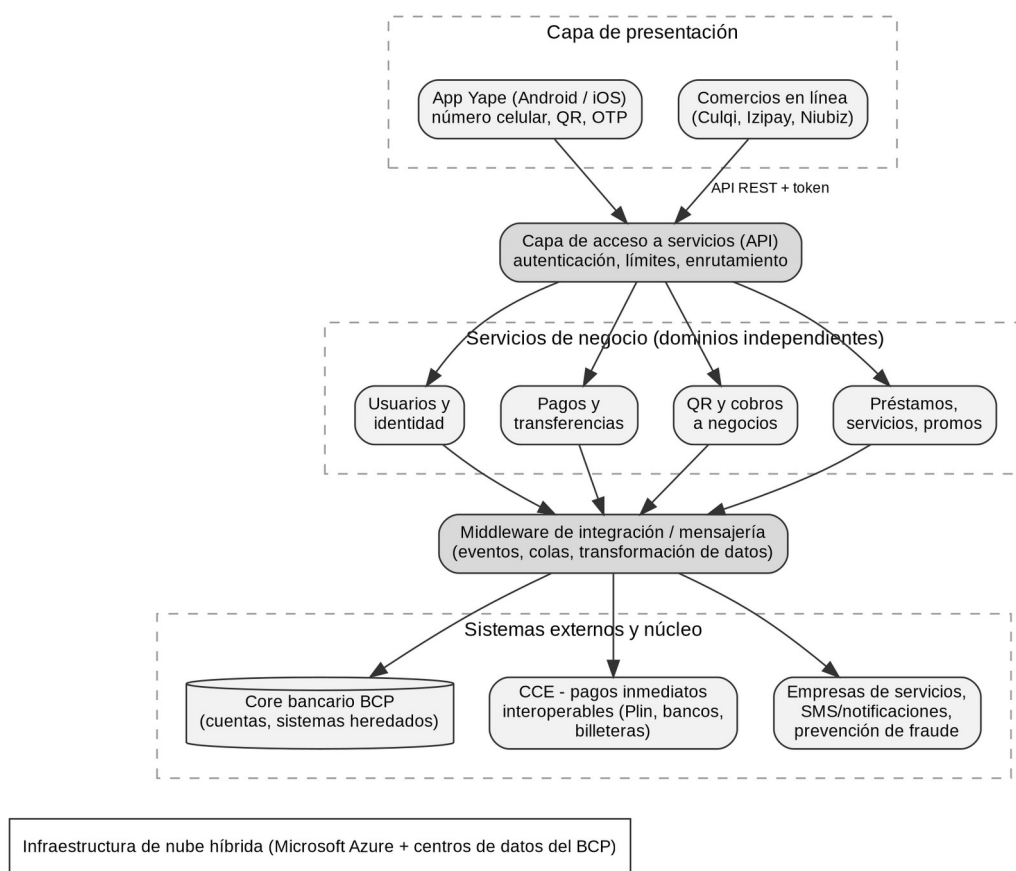
5.4. Patrón arquitectónico

Con la información disponible, la arquitectura de Yape se puede describir como **microservicios detrás de una capa de API, con integración por eventos, desplegada sobre nube híbrida**, y conectada al resto del sistema financiero mediante un patrón **concentrador y radios** (*hub and spoke*) cuyo centro es la CCE. Esta descripción es una reconstrucción razonada, no un diagrama oficial, y se sustenta en tres hechos:

- **Productos independientes.** Pagos, QR para negocios, préstamos, promociones, pago de servicios y compras en línea evolucionan a ritmos distintos. En microservicios, cada dominio se despliega y escala por separado (Richards y Ford, 2020); esto encaja con una app que suma funciones con frecuencia.
- **Volumen y picos.** Con decenas de millones de transacciones diarias y picos en quincenas o fechas festivas, conviene escalar solo los servicios más usados, algo que la nube híbrida del BCP permite.
- **Integración con muchos terceros.** Bancos, la CCE, empresas de servicios y pasarelas se conectan por APIs y mensajes, no por conexiones punto a punto, tal como recomienda la sesión para reducir el acoplamiento.

Figura 11

Arquitectura referencial de Yape



Nota. Elaboración propia. Representación referencial reconstruida a partir de fuentes públicas (Credicorp, 2026; Culqi, s. f.; Gestión, s. f.; Microsoft News Center Latinoamérica, 2024); no es un diagrama oficial de Yape.

5.5. Otras características

- **Interoperabilidad regulada.** La Circular N.º 0024-2022-BCRP obligó a las billeteras y a los bancos a interoperar: los pagos entre Yape y Plin debían estar disponibles a más tardar el 31 de marzo de 2023, con criterios de cobro transparentes y no discriminatorios (Latamfintech, 2022). En una segunda fase se integraron emisores de dinero electrónico y cooperativas, y en una tercera se habilitó a fintech, bigtech y operadoras de telecomunicaciones como participantes indirectos (Gestión, s. f.).
- **Seguridad.** Las compras por Internet usan códigos de aprobación de un solo uso con vigencia de dos minutos; las integraciones directas con la API exigen cumplir PCI DSS (Culqi, s. f.). Los participantes indirectos deben registrarse ante el BCRP e informar a la CCE los cambios en su plataforma y los incidentes operativos (Gestión, s. f.).
- **Disponibilidad.** Los pagos inmediatos funcionan todos los días y a toda hora, por lo que la plataforma necesita alta disponibilidad y tolerancia a fallas.

- **Escalamiento regional.** La operación en Bolivia desde 2023 muestra que la plataforma se puede replicar en otro país adaptando la integración con el sistema financiero local (Infobae, 2025).

5.6. Relación con el curso

Yape es un ejemplo real de sistema de información integrado. Su valor no está en una sola aplicación, sino en conectar de forma confiable la app, el banco, otras billeteras, los comercios y las empresas de servicios. Los conceptos de la sesión aparecen en cada una de esas conexiones: APIs, intermediarios, transformación de datos, colas y seguridad. Para el proyecto hotelero del grupo, la lección es directa: el módulo de CRM deberá exponer sus datos mediante APIs bien definidas y comunicarse con los demás módulos a través de una capa de integración, no mediante conexiones directas entre bases de datos.

6. Caso 5: ensayo

Infraestructura que mejora vidas: lo que las APP necesitan además de capital

Obra analizada: Lembo, C., Mejdalani, A., y Suárez-Alemán, A. (2025). *PPP Américas 2025*.

Perú: Alianzas que mejoran vidas (capítulos 1 y 2). Banco Interamericano de Desarrollo.

6.1. Introducción

América Latina y el Caribe necesita aumentar alrededor de un 70 % su inversión en infraestructura para cerrar sus brechas, cerca de 250 000 millones de dólares al año, en un contexto en el que la deuda de la región ya llega al 117 % de su PIB (Lembo et al., 2025). Con ese punto de partida, el documento que el BID preparó para el encuentro PPP Américas 2025, realizado en Lima, presenta las asociaciones público-privadas (APP) como una forma de movilizar capital privado sin perder de vista a las personas que usan los servicios. Este ensayo resume los capítulos 1 y 2, ofrece una opinión sobre sus argumentos y los relaciona con los sistemas de información integrados y con nuestra carrera.

6.2. Síntesis del capítulo 1: el poder de las APP en la infraestructura social

El primer capítulo se ocupa de la infraestructura social: agua y saneamiento, educación y salud. Los autores recuerdan que solo el sector salud necesita al menos 100 000 millones de dólares para adecuar hospitales y equipos, y que el 35 % de los estudiantes de la región asiste a escuelas mal equipadas. También identifican tres tipos de desafíos. El financiero surge porque el usuario de estos servicios, por lo general, no paga su costo completo, así que el Estado debe garantizar el pago al socio privado. El regulatorio aparece porque muchas leyes de APP fueron pensadas para carreteras; en Colombia, por ejemplo, no se han hecho APP de agua y saneamiento por esa razón. El político se debe a que las comunidades suelen ver a las APP como una privatización (Lembo et al., 2025).

El capítulo analiza seis casos. En el Perú, Provisur llevó agua y saneamiento a Punta Hermosa, Punta Negra, San Bartolo y Santa María del Mar con una inversión de 100 millones de dólares y la primera planta desalinizadora de uso público del país, de 400 litros por segundo; sin embargo, la falta de comunicación inicial con los vecinos generó desconfianza y retrasó el proyecto. En México, la planta de Atotonilco trata las aguas residuales de más de 12 millones de

personas y usa el biogás del proceso para producir energía. En Uruguay, el proyecto PPP Educativa 3 incluye 15 escuelas y 27 centros de atención a la infancia con un plan formal de participación de actores sociales. En Brasil, Inova Belo Horizonte construyó 51 escuelas con más de 60 indicadores de desempeño y atendía a 42 000 alumnos en 2017, frente a los 25 000 previstos. En Bogotá, el Hospital de Bosa se construyó en tres años sin retrasos gracias a la metodología BIM y exigió que el 30 % de la mano de obra fuera local. Por último, el Hospital do Subúrbio de Salvador vinculó el pago al concesionario con indicadores de desempeño auditados (Lembo et al., 2025).

6.3. Síntesis del capítulo 2: proyectos resilientes y centrados en las personas

El segundo capítulo sostiene que una APP no solo debe entregar una obra, sino también reducir desigualdades y resistir los efectos del clima. Para lo primero propone involucrar a las comunidades desde la planificación mediante consultas que no sean simbólicas, incluir cláusulas de contratación local, definir indicadores de inclusión y acompañar los proyectos después de su puesta en marcha. Para lo segundo presenta datos preocupantes: unos 155 millones de personas de la región, cerca del 24 % de la población, viven en situación de vulnerabilidad climática, y solo en 2023 los desastres causaron daños por 21 000 millones de dólares, dos tercios de ellos por tormentas como el huracán Otis (Lembo et al., 2025).

La respuesta que proponen es incorporar el riesgo climático en todo el ciclo del proyecto, ya que cada dólar invertido en resiliencia genera unos cuatro dólares de beneficio. Los ejemplos son la política nacional de APP de Jamaica, que exige estudios de riesgo climático y usa la herramienta C-CORAL; el Tren Interciudades São Paulo–Campinas, cuyo contrato obliga a analizar las lluvias futuras con escenarios del IPCC en lugar de basarse solo en datos históricos; y las concesiones forestales de Brasil, que desde 2023 pueden recibir pagos por servicios ambientales. El capítulo termina con recomendaciones: marcos normativos estables, participación de todos los actores, análisis de riesgos climáticos y tecnologías sostenibles (Lembo et al., 2025).

6.4. Opinión

El mayor acierto del documento es cambiar la pregunta. En lugar de preguntar solo cuánto dinero privado se puede atraer, pregunta si la obra mejora la vida de quienes la usan. Los casos respaldan esa idea con datos concretos, y la lección más repetida no es técnica ni financiera, sino de comunicación: Provisur se retrasó por no informar a tiempo a los vecinos, mientras que Atotonilco y Bosa evitaron conflictos porque informaron desde el inicio. Como futuros ingenieros solemos pensar que un buen diseño basta; estos casos muestran que un proyecto técnicamente correcto puede fracasar si las personas no confían en él.

También encontramos debilidades. La primera es un sesgo de selección: todos los casos son exitosos, y algunos, como PPP Educativa 3, siguen en construcción, por lo que sus "lecciones" se basan en documentos de planificación y no en resultados. Un documento para la discusión sería más útil si analizara también proyectos que fallaron. La segunda es que casi no se discuten los riesgos fiscales. Si el Estado debe garantizar los pagos de una APP social, esas garantías se convierten en compromisos futuros para el presupuesto público, y la experiencia regional muestra que muchas concesiones terminan renegociadas en condiciones desfavorables para el Estado (Guasch, 2004). La tercera es que la medición del impacto social se remite al capítulo 3, cuando es justamente lo que permitiría comprobar que estas alianzas "mejoran vidas".

6.5. Relación con los sistemas de información integrados y con la carrera

Aunque el documento trata sobre infraestructura, casi todas sus recomendaciones dependen de información integrada. El pago por desempeño del Hospital do Subúrbio, los más de 60 indicadores de Inova BH, la veeduría ciudadana de Bosa o el monitoreo climático que exige el Tren Interciudades solo funcionan si los datos del concesionario, de la entidad pública y de los ciudadanos se comparten de manera confiable y verificable. En términos del curso, cada APP es un sistema con varios actores que deben intercambiar información sin depender unos de otros, que es exactamente el problema que resuelven el middleware y las plataformas de integración de la sesión S04. Bosa lo muestra con claridad: BIM, inteligencia artificial y realidad virtual permitieron supervisar la obra y cumplir los plazos.

Para Ingeniería de Sistemas e Inteligencia Artificial esto abre un campo concreto: plataformas de monitoreo de contratos, tableros públicos de indicadores, modelos que usen proyecciones climáticas para anticipar riesgos y canales digitales para recoger la opinión de las comunidades. También deja una lección para nuestro proyecto hotelero: igual que en una APP, el módulo de CRM solo aportará valor si sus datos se integran con los demás módulos y si las personas que lo usarán participan desde su diseño.

6.6. Conclusiones

- Las APP pueden ayudar a cerrar la brecha de infraestructura social de la región, pero su éxito depende tanto del diseño contractual como de la confianza de las comunidades.
- La comunicación temprana y la participación ciudadana aparecen en los casos como factores decisivos para cumplir plazos y evitar conflictos.
- La resiliencia climática debe incluirse en los contratos con datos prospectivos, porque la región es muy vulnerable y reconstruir es más caro que prevenir.
- El documento se beneficiaría de analizar casos fallidos y los riesgos fiscales de las garantías públicas.
- Medir, monitorear y transparentar el desempeño de una APP es, en el fondo, un problema de integración de información, en el que los ingenieros de sistemas tenemos un papel directo.

7. Enlace y contenido de la solución

Todos los archivos solicitados se encuentran en el siguiente repositorio público de GitHub:

<https://github.com/JeffryRU/SII-S04-Middleware>

Tabla 9

Contenido del repositorio

Ruta	Contenido
README.md	Descripción general, integrantes e instrucciones de ejecución.
mensajeria/protocolo.py	Formato de las tramas (longitud + JSON).
mensajeria/servidor.py	Servidor intermediario de mensajería (caso 1).
mensajeria/cliente.py	Cliente con interfaz gráfica (caso 1).
excel-access/VERDULEROS.mdb	Base de datos de Access proporcionada.
excel-access/ Conexion_Excel_VERDULEROS.xlsx	Libro de Excel conectado a la base de datos, con tabla y gráfico dinámicos (caso 2).
excel-access/capturas/	Capturas del procedimiento de conexión.
diccionario/diccionario.py	Formulario de consulta de palabras (caso 3).
docs/	Este informe en PDF.

Nota. Elaboración propia.

Requisitos: Python 3.8 o superior (solo la biblioteca estándar) y Microsoft Excel 2016 o posterior. Las instrucciones detalladas de ejecución están en el archivo README.md y en las secciones 2.6, 3.3 y 4.3 de este informe.

Referencias

- Credicorp. (2026, 15 de mayo). *Yape alcanzó 16.4 millones de usuarios activos mensuales consolidando su cobertura a nivel nacional*. <https://grupocredicorp.com/yape-alcanzo-16-4-millones-de-usuarios-activos-mensuales-consolidando-su-cobertura-a-nivel-nacional/>
- Culqi. (s. f.). *Cargos únicos – Yape* [Documentación técnica]. <https://docs.culqi.com/es/documentacion/pagos-online/cargo-unico/tokens-yape>
- Curry, E. (2004). Message-oriented middleware. En Q. H. Mahmoud (Ed.), *Middleware for communications* (pp. 1–28). John Wiley & Sons.
- Gestión. (s. f.). *BCRP deja todo listo para que bigtech y telcos interoperen con Yape y Plin*. <https://gestion.pe/tu-dinero/finanzas-personales/bcrp-deja-todo-listo-para-que-bigtech-y-telcos-interoperen-con-yape-y-plin-interoperabilidad-billeteras-digitales-noticia/>
- Guasch, J. L. (2004). *Granting and renegotiating infrastructure concessions: Doing it right*. Banco Mundial.
- Infobae. (2025, 3 de junio). *Yape de Perú conquista Bolivia: la app del BCP tiene más de 3 millones de usuarios y es la favorita de los bolivianos*. <https://www.infobae.com/peru/2025/06/03/yape-de-peru-conquista-bolivia-la-app-del-bcp-tiene-mas-de-3-millones-de-usuarios-y-es-la-favorita-de-los-bolivianos/>
- Latamfintech. (2022). *Billeteras digitales Yape y Plin deberán ser interoperables según el Banco Central de Reserva del Perú*. <https://www.latamfintech.co/articles/billeteras-digitales-yape-y-plin-deberan-ser-interoperables-segun-el-banco-central-de-reserva-del-peru>
- Lembo, C., Mejdalani, A., y Suárez-Alemán, A. (2025). *PPP Américas 2025. Perú: Alianzas que mejoran vidas* (Documento para discusión N.º IDB-DP-0177). Banco Interamericano de Desarrollo. <https://doi.org/10.18235/0013483>
- Mendoza Corpus, C. A. (2026). *Sesión 04: Plataformas middleware* [Diapositivas]. Curso Sistemas de Información Integrados, Universidad Privada Antenor Orrego.

Microsoft. (s. f.-a). *Power Query Access database connector*. Microsoft Learn.

<https://learn.microsoft.com/en-us/power-query/connectors/access-database>

Microsoft. (s. f.-b). *What is ODBC?* Microsoft Learn.

<https://learn.microsoft.com/en-us/sql/odbc/reference/what-is-odbc>

Microsoft News Center Latinoamérica. (2024, 11 de noviembre). *BCP invierte más de S/*

2,500 millones para modernizar su infraestructura tecnológica de la mano de

Microsoft y Kyndryl. [https://news.microsoft.com/es-xl/bcp-invierte-mas-de-s-2500-](https://news.microsoft.com/es-xl/bcp-invierte-mas-de-s-2500-millones-para-modernizar-su-infraestructura-tecnologica-de-la-mano-de-microsoft-y-kyndryl/)

[millones-para-modernizar-su-infraestructura-tecnologica-de-la-mano-de-microsoft-y-](https://news.microsoft.com/es-xl/bcp-invierte-mas-de-s-2500-millones-para-modernizar-su-infraestructura-tecnologica-de-la-mano-de-microsoft-y-kyndryl/)

[kyndryl/](https://news.microsoft.com/es-xl/bcp-invierte-mas-de-s-2500-millones-para-modernizar-su-infraestructura-tecnologica-de-la-mano-de-microsoft-y-kyndryl/)

Richards, M., y Ford, N. (2020). *Fundamentals of software architecture: An engineering approach*. O'Reilly Media.

Universidad Tecnológica del Perú. (2024, 17 de mayo). *Conoce la historia de Yape*.

<https://www.utp.edu.pe/blog/la-historia-de-yape>

Yape. (s. f.). *¿Cómo puedo tener la opción de compras por internet (código de aprobación)*

Yape en la página de mi comercio? Centro de Ayuda Yape.

[https://www.yape.com.pe/preguntas-frecuentes/compras-por-internet/como-puedo-](https://www.yape.com.pe/preguntas-frecuentes/compras-por-internet/como-puedo-tener-la-opcion-de-compras-por-internet-codigo-de-aprobacion-yape-en-l)

[tener-la-opcion-de-compras-por-internet-codigo-de-aprobacion-yape-en-l](https://www.yape.com.pe/preguntas-frecuentes/compras-por-internet/como-puedo-tener-la-opcion-de-compras-por-internet-codigo-de-aprobacion-yape-en-l)